Public Key Crypto notes - 07

By contributors

March 23. 2026

1 Schnorr-signature recap

- Gen:
  - Input: 1^n
  - Output: $pk = (p, g, q, h = g^x), sk = x$
- Sig:
  - Input: sk, m
  - Output: (I, s), where:

$$k \in_R \mathbb{Z}_q, \tag{1}$$
$$I = g^k, \tag{2}$$
$$r = H(I, m), \tag{3}$$
$$s = rx + k \tag{4}$$

- Ver:
  - Input: $pk, m, (I, s)$
  - Output: Compute $r = H(I||m)$ and check whether $I = g^s \cdot h^{-r}$

This scheme is patented. Instead of this scheme, we mostly use DSA (made by NIST).

2 Digital Signature Algorithm (DSA)

- Designed by NIST.
- Let $H : \{0,1\}^* \to \mathbb{Z}_q, F : \{0,1\}^* \to \mathbb{Z}_q$ be two functions. Consider the following three algorithms:
- Gen:
  - Input: 1^n
  - Output: Public key $pk = (p, g, q, h = g^x)$, Secret key: $sk = x$
- Sig:
  - Input: m, sk

  - Output: (r, s)
  - $k \in_R \mathbb{Z}_q, r = F(g^k), s = k^{-1}(H(m) + xr) \pmod q$

- Ver:
  - Input: $m, (r, s), pk$
  - Output: $r \stackrel{?}{=} F(g^{H(m)\cdot s^{-1}} \cdot h^{r\cdot s^{-1}})$

- Correctness:

$$g^{H(m)\cdot s^{-1}} \cdot h^{r\cdot s^{-1}} = \tag{5}$$
$$g^{H(m)\cdot s^{-1}+x\cdot r\cdot s^{-1}} = \tag{6}$$
$$g^{s^{-1}(H(m)+x\cdot h)} = \tag{7}$$
$$g^{\frac{k}{H(m)+xr}\cdot(H(m)+xr)} = g^k \tag{8}$$
$$F(g^k)\checkmark \tag{9}$$

Theorem 2.1 (Security of the DSA scheme)**.** If H and F are modelled by a random oracle and the discrete logarithm problem is hard, then DSA is secure.

Remark 2.1. In practice, $F(x) = x \pmod q$, this means that the "random oracle" is not really random.

Remark 2.2. If the same k is used multiple times, then one can obtain $sk = x$. It is important to use a different k for each iteration.
Indeed:

$$s_1 = k^{-1}(H(m_1) + x + r) \tag{10}$$
$$s_2 = k^{-1}(H(m_2) + x + r) \tag{11}$$
$$s_1 - s_2 = k^{-1}(H(m_1) - H(m_2)) \to k \text{ can be computed} \tag{12}$$
$$\to x \text{ can also be computed.} \tag{13}$$

Note: Playstation 3: All k used for updates were the same.[1]

2.1 Parameter selection

- p is large, $\approx 3000 - 4000$ bits,
- q is medium size (size ≈ 256 bits),
- $\to$ signature size: 2×256 bits ≈ 500 bits.

[1] https://deeprnd.medium.com/decoding-the-playstation-3-hack-unraveling-the-ecdsa-random-generator-flaw-e9074a51

3 Further digital signatures using the discrete log problem

General form of signatures:

- Given $a, b, c : ak = b + cx \pmod{q}$
- Verification: $g^{ak} \stackrel{?}{=} g^b \cdot g^{xc}$
- Examples:
  1. Schnorr:
     - $a = -1$
     - $b = s$
     - $c = -r$
  2. DSA:
     - $a = s$
     - $b = H(m)$
     - $c = (g^k \pmod{p}) \pmod{q}$
  3. ElGamal:
     - $a = s$
     - $b = h(m)$
     - $c = -r \pmod{p-1}$

4 Exotic protocols

4.1 Blind signature

- Given: signer S and user U.
- U has a message m and wants S to sign it without telling m to S.

4.2 Blind RSA signature

- The signer has plain RSA keys, where $pk = (N, e), sk = d$.
- U has message $m \in \mathbb{Z}_N^*$, blinding parameter $r \in_R \mathbb{Z}_N^*$, and sends $m' = m \cdot r^e \pmod{N}$ to S.
- S signs m', and sends $s' = m'^d \pmod{N}$ to U.
- So $s = s' \cdot r^{-1} \pmod{N} = m^d (r^e)^d r^1 = m^d$

Remark 4.1. The value r is the blinding parameter. If it is chosen randomly, then m' cannot leak any information about m.

4.3 Blind Schnorr signature

- S chooses $k \in_R \mathbb{Z}_q$, sends $I = g^k$ to U.
- U chooses blinding parameters $\alpha, \beta \in_R \mathbb{Z}_q$.
- U calculates:

$$I' = I \cdot g^\alpha \cdot h^\beta \tag{14}$$
$$r' = H(I'||m) \tag{15}$$
$$r = r' + \beta \tag{16}$$

  and sends r to S.
- S calculates $s = rx + k$ and sends it to U.
- U calculates $s' = s + \alpha$
- The signature is $sig = (I', s')$

Correctness:

$$g^s \stackrel{?}{=} I \cdot h^r = \tag{17}$$
$$g^{s'-\alpha} \stackrel{?}{=} I' \cdot g^{-\alpha} \cdot h^{-\beta} \cdot g^{r'+\beta} = \tag{18}$$
$$g^{s'} \stackrel{?}{=} I' \cdot h^{r'} \tag{19}$$

4.4 Oblivious transfer

There are two parties: sender A and receiver B. A wants to send a message (out of two) to B, such that:

- B can only receive one of the messages.
- A should *not* learn which message B received.
- Formally:
  - Two messages: m_0, m_1
  - B chooses $b \in_R \{0, 1\}$ and receives m_b

4.4.1 Protocol 1 (Even, Goldreich, Lempel)

- Based on RSA.
- A has two messages: m_0, m_1 and generates RSA keys $pk = (N, e), sk = d$.
- A chooses $x_0, x_1 \in_R \mathbb{Z}_N^*$ and sends them to B along with pk.
- B chooses $b \in_R \{0, 1\}$ and $k \in_R \mathbb{Z}_N^*$, and computes $v = (x_b + k^e) \pmod N$ and sends v (also called a *pre-envelope*) to A.

- A computes:

$$k_0 = (v - x_0)^d, \tag{20}$$
$$k_1 = (v - x_1)^d, \tag{21}$$
$$m_0' = m_0 + k_0, \tag{22}$$
$$m_1' = m_1 + k_1, \tag{23}$$
$$\tag{24}$$

  and sends m_0', m_1' to B.

- B computes $m_b = m_b' - k$

But B cannot compute $k_{1-b} = (v - x_{1-b})^d$

Remark 4.2. This is a one-out-of-two oblivious transfer, but there are 1-out-of-n or m-out-of-n oblivious transfer schemes out there also.